



BEE: Intervention-Adaptive Real-World Reinforcement Learning with Vision-Language-Action Models

Weihui Zhao^{1,2}, Xiaohan Yan^{2,†}, Zunian Wan², Xuan Du², Zhaozhan Chi², Jianbo Mao²
 Ruipu Wu², Rushuai Yang², Houlin Li², Shukai Yang², Jing Wu², Yuxiang Yan²
 Yongcheng Liu², Chuankang Li², Guanghui Ren², Wei Shan², Maoqing Yao²

¹South China University of Technology

²AgiBot

[†]Corresponding Author

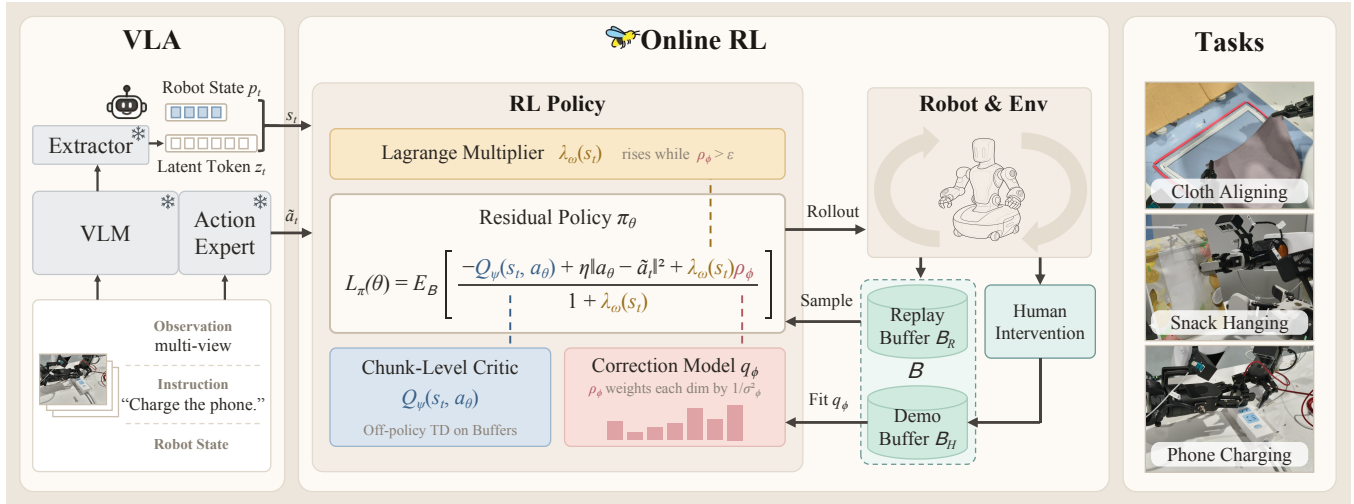


Fig. 1: **Overview of BEE.** BEE converts human corrections of VLA proposals into an uncertainty-aware constraint on policy optimization. *Left:* a frozen VLA produces an action proposal $\hat{\mathbf{a}}_t$ while an extractor maps the VLA’s internal features to a latent token that, with the robot state p_t , forms the RL state s_t . *Middle:* rather than imitating interventions, **BEE** distills them via a Correction Model q_ϕ into an adaptive constraint ρ_ϕ that weights each action dimension by the inverse of its predicted correction variance σ_ϕ^2 . The Residual Policy π_θ refines $\hat{\mathbf{a}}_t$ into $\mathbf{a}_\theta$ by optimizing $L_\pi(\theta)$, which balances Q_ψ and a VLA anchor (η) against ρ_ϕ weighted by a Lagrange multiplier $\lambda_w(s_t)$. *Right:* we evaluate **BEE** on three real-world tasks.

Abstract—Vision-language-action (VLA) models handle long-horizon manipulation, yet success hinges on a few precision-critical phases where millimeter-scale errors undo all prior progress. Online reinforcement learning (RL) can optimize exactly these actions, but free exploration is far too costly on real robots, which makes human corrections indispensable. However, existing online RL methods for VLAs either cannot incorporate such corrections or fold them into undifferentiated supervision. Yet human corrections are not uniformly noisy but reliable along some action dimensions and variable along others. Building on this, we introduce BEE, an intervention-adaptive framework for real-world RL on a frozen VLA that lets the policy go BEYond Expert imitation. We formulate human corrections not as actions to reproduce but as evidence about a constraint: a Correction Model predicts how a human would correct a given VLA proposal and how consistent the correction is along each action dimension. This predicted consistency sets the per-dimension tightness of a constraint on policy optimization. Where corrections are consistent the policy stays close to the human, and where they vary, the constraint relaxes. We evaluate BEE on three real-world manipulation tasks and one LIBERO-Pro simulation task at a matched online-data budget. BEE attains the highest success rate on every task, 91.2% on average against 57.5% for RLT and 42.1% for DSRL,

and the lowest human intervention rate on all real-world tasks.

I. INTRODUCTION

Vision-language-action (VLA) models have emerged as capable generalist controllers for robotic manipulation, leveraging broad behavioral priors and semantic representations to execute long-horizon, multi-stage tasks from visual observations and language instructions [1], [2], [3], [4], [5], [6]. Yet task success often hinges on a few precision-critical phases such as alignment and insertion, where millimeter-scale pose errors invalidate all preceding progress. Since human demonstrations are usually noisy and rarely cover these failure modes densely, even a capable VLA may stall near the final contact. This brittleness makes online reinforcement learning (RL) attractive. By interacting with the target environment, RL can directly optimize the actions that determine task success [7], [8]. On real robots, however, every rollout costs time and hardware life, and sparse rewards coupled with high-dimensional action chunks limit unconstrained exploration, making targeted human corrections a

valuable but scarce resource.

However, existing online RL methods for VLAs either provide no mechanism for human corrections or use them without distinction. DSRL [9] steers the frozen VLA by searching its latent noise space and has no interface for human input at all. The recent RL Token (RLT) framework [10] stores human interventions and autonomous rollouts in a shared replay buffer. When the human intervenes, RLT replaces the VLA proposal stored for that transition with the human correction, so its actor is trained with human corrections in place of VLA proposals on intervention samples, whereas its proposal input always comes from the VLA at deployment. Prior work such as SiLRI [11] adjusts the overall imitation strength with a state-wise uncertainty estimate but still treats all action dimensions uniformly. Yet human corrections are not uniformly noisy but reliable along some action dimensions and variable along others, which opens the door to constraining the policy selectively rather than uniformly.

To this end, we introduce **BEE**, an intervention-adaptive framework for VLA-anchored real-world RL that retains the VLA proposal that prompted each human correction and models the correction residual relative to it. **BEE** converts these corrections into an uncertainty-aware constraint on policy optimization, allowing the policy to go **BE**yond Expert imitation. As illustrated in Figure 1, **BEE** draws two outputs from a single frozen VLA. An extractor converts the features of the VLA into a latent token that supplies task-relevant input to online RL. The action expert produces an action chunk that is retained as a deployment-time proposal, which a lightweight chunk-level *Residual Policy* then refines into the action executed by the robot. Rather than imitating human interventions directly, **BEE** uses them to train a separate *Correction Model*. Conditioned on the current state and on the VLA proposal, the Correction Model predicts a distribution over the correction residual, from which a mean and an uncertainty are obtained for each action dimension. Adding the predicted mean to the VLA proposal yields the corrected action, which serves as the center of a correction constraint on the policy, whereas the predicted uncertainty measures how consistent human corrections are in each action dimension. RL then optimizes the Residual Policy subject to this constraint, which keeps the policy close to the corrected action along consistent dimensions while allowing the constraint to relax along variable ones. Overall, our contributions are as follows:

- We introduce **BEE**, an intervention-adaptive framework for VLA-anchored real-world RL that converts human interventions into a proposal-conditioned correction prior while preserving the long-horizon competence of the frozen VLA.
- We develop an uncertainty-aware constrained policy optimization, which constrains each action dimension according to the reliability of human corrections in that dimension.
- We evaluate **BEE** on three real-world manipulation tasks and on a LIBERO-Pro simulation task. **BEE** attains the

highest success rate on every task and the lowest human intervention rate on all real-world tasks.

II. RELATED WORK

A. RL for Vision-Language-Action Models

Reinforcement learning increasingly extends pretrained VLA policies beyond their demonstration data, through large-scale simulated rollouts [12], [13], [14], preference alignment [15], or distillation of task-specific RL controllers [16]. On real hardware, ConRFT [17] trains a consistency-based action head offline and then online, and iRe-VLA [18] alternates supervised and actor-critic updates, yet both fine-tune the VLA rather than keep its output as a fixed proposal for online correction. DSRL [9] instead freezes the base policy and steers it toward higher return by searching its latent noise space.

RLT [10] freezes the VLA and trains a chunk-level actor-critic that takes a latent token together with the explicit action proposal of the VLA as input. Because the learned policy is kept close to this proposal, the resulting update amounts to value maximization regularized toward a behavior prior [19], [20], with the frozen VLA serving as that prior. **BEE** builds on this interface with a residual policy [21], [22] but focuses on how human corrections should guide policy improvement.

B. Human-in-the-Loop RL

Human-in-the-loop RL improves the exploration efficiency of real-world RL by allowing a human operator to guide the policy during online interaction. These methods differ mainly in when the human takes over and in how the resulting data are reused. DAgger [23] relabels on-policy states with actions queried from an always-available human, whereas HG-DAgger [24] and ThriftyDAgger [25] gate takeovers by human judgment or by a risk budget. Intervention-weighted schemes instead upweight corrective segments [26], [27], and other formulations treat the intervention signal as reward or value information rather than as an imitation target [28], [29]. Building on off-policy learning from mixed offline and online data [30], SERL [31] and RL-100 [8] provide practical training pipelines, and HIL-SERL [7] further incorporates demonstrations and human takeovers at failure-prone states, which supplies corrective transitions that autonomous exploration rarely encounters. Such targeted assistance is valuable when robot time is scarce, yet these pipelines train task-specific visuomotor networks [32], [33] and therefore leave the competence of a pretrained VLA unused.

SiLRI [11] accounts for noisy human interventions through a state-conditioned human behavior model and a state-wise Lagrange multiplier that balances return maximization against imitation. The constraint of SiLRI, however, is a single scalar bound per state, so it cannot express how the reliability of a human correction varies from one action dimension to the next. In contrast, **BEE** predicts a per-dimension correction distribution in the spirit of heteroscedastic uncertainty estimation [34], [35], so that the

resulting Mahalanobis constraint adapts across action dimensions, while the overall constraint strength is still set by a state-dependent multiplier trained by dual ascent [36].

III. PRELIMINARIES

A. Chunk-Level RL over a VLA Interface

We formulate task execution as a chunk-level RL problem. At each chunk boundary t , the agent observes a state $\mathbf{s}_t$ and emits an action chunk $\mathbf{a}_t \in \mathbb{R}^D$, which concatenates K primitive actions each of dimension d ($D=Kd$). The environment returns a reward r_t , and we maximize the expected discounted return $J(\pi) = \mathbb{E}_\pi[\sum_t \gamma^t r_t]$ with discount factor $\gamma \in (0, 1)$.

Following RLT [10], we ground this state and action space in a frozen VLA while retaining its action proposal. At each boundary t , the frozen VLA π_{vla} maps the current observation $\mathbf{o}_t$, language instruction g , and proprioceptive state $\mathbf{p}_t$ to an action proposal $\tilde{\mathbf{a}}_t \sim \pi_{\text{vla}}(\cdot | \mathbf{o}_t, g, \mathbf{p}_t)$. An Extractor converts the VLA's internal features into a latent token $\mathbf{z}_t$, giving the RL state $\mathbf{s}_t = [\mathbf{z}_t \| \mathbf{p}_t]$. The pair $(\mathbf{s}_t, \tilde{\mathbf{a}}_t)$ is the fixed VLA-RL interface, with the critic evaluating chunks at $\mathbf{s}_t$ and the policy additionally conditioning on $\tilde{\mathbf{a}}_t$.

A chunk-level critic $Q_\psi(\mathbf{s}_t, \mathbf{a}_t)$ evaluates an entire chunk rather than a single step [10], trained off-policy on transitions sampled from a buffer $\mathcal{B}$ with the K -step Bellman target

$$y_t = R_t + \gamma^K (1 - \delta_t) Q_\psi(\mathbf{s}_{t+K}, \mathbf{a}_{t+K}), \quad (1)$$

where $R_t = \sum_{j=0}^{K-1} \gamma^j r_{t+j}$ is the discounted chunk return, δ_t indicates termination within the chunk, $\mathbf{a}_{t+K}$ is the next chunk produced by the current policy, and Q_ψ is a target critic. The critic minimizes the Bellman error

$$L_Q(\psi) = \mathbb{E}_\mathcal{B}[(Q_\psi(\mathbf{s}_t, \mathbf{a}_t) - y_t)^2]. \quad (2)$$

B. Learning from Human Interventions

In human-in-the-loop RL [7], a human correction $\mathbf{a}_t^H$ overrides the policy action and becomes the executed action $\mathbf{a}_t$. All online transitions, autonomous or human-corrected, go to the replay buffer $\mathcal{B}_R$, and human-corrected chunks are additionally copied into a demonstration buffer $\mathcal{B}_H$. Each training batch is drawn in equal parts from the two, and $\mathcal{B}$ denotes this training distribution. The VLA and the encoder stay frozen throughout, and rollout collection, human interventions, and off-policy updates interleave at each chunk boundary. **BEE** builds on this off-policy critic and its two buffers, turning human interventions into an uncertainty-aware constraint on the policy.

IV. METHOD

A. Problem Formulation

At each chunk boundary, the frozen VLA-RL interface provides a state-proposal pair $(\mathbf{s}_t, \tilde{\mathbf{a}}_t)$, where $\mathbf{s}_t = [\mathbf{z}_t \| \mathbf{p}_t]$. The proposal is an additional input to the online learner. We do not propagate gradients through the frozen VLA.

A deterministic Residual Policy predicts an action residual and produces $\mathbf{a}_\theta = \tilde{\mathbf{a}}_t + \Delta_\theta^\pi(\mathbf{s}_t, \tilde{\mathbf{a}}_t)$. Meanwhile, a proposal-conditioned Correction Model represents how a human

would correct the same proposal. Let $\boldsymbol{\mu}_\phi$ and $\boldsymbol{\Sigma}_{\phi,t} = \text{diag}(\boldsymbol{\sigma}_\phi^2)$ denote its predicted mean residual and diagonal covariance, respectively. They define the predicted corrected action $\hat{\mathbf{a}}_t^H = \tilde{\mathbf{a}}_t + \boldsymbol{\mu}_\phi$ and the normalized Mahalanobis deviation

$$\rho_\phi = \frac{1}{D} (\mathbf{a}_\theta - \hat{\mathbf{a}}_t^H)^\top \boldsymbol{\Sigma}_{\phi,t}^{-1} (\mathbf{a}_\theta - \hat{\mathbf{a}}_t^H). \quad (3)$$

Thus, consistent human corrections define narrow feasible directions, whereas variable corrections leave more freedom for return-driven improvement.

Our objective is to improve task return without allowing the learned policy to depart arbitrarily from this correction-informed feasible set. We formulate policy learning as

$$\begin{aligned} \min_{\theta} \quad & \mathbb{E}_\mathcal{B}[-Q_\psi(\mathbf{s}_t, \mathbf{a}_\theta) + \eta \|\mathbf{a}_\theta - \tilde{\mathbf{a}}_t\|_2^2] \\ \text{s.t.} \quad & \rho_\phi \leq \varepsilon, \end{aligned} \quad (4)$$

where the constraint is evaluated on state-proposal pairs sampled from $\mathcal{B}$, and $\varepsilon > 0$ is fixed. The predicted correction mean centers the feasible set, and its variances determine the set's dimension-wise geometry. Since q_ϕ is Gaussian, for a fixed Correction Model, ρ_ϕ is proportional to the actor-dependent term of $-\log q_\phi(\mathbf{a}_\theta - \tilde{\mathbf{a}}_t | \mathbf{s}_t, \tilde{\mathbf{a}}_t)$.

B. Learning the Correction Distribution

Each human-corrected sample copied into the demonstration buffer $\mathcal{B}_H$ retains both the VLA proposal $\tilde{\mathbf{a}}_t$ and the human correction $\mathbf{a}_t^H$, rather than replacing the proposal with the correction. With correction residual $\Delta_t^H = \mathbf{a}_t^H - \tilde{\mathbf{a}}_t$, the Correction Model predicts

$$q_\phi(\Delta_t^H | \mathbf{s}_t, \tilde{\mathbf{a}}_t) = \mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{s}_t, \tilde{\mathbf{a}}_t), \boldsymbol{\Sigma}_{\phi,t}). \quad (5)$$

It is trained only on $\mathcal{B}_H$ by negative log-likelihood,

$$\mathcal{L}_{\text{corr}}(\phi) = -\mathbb{E}_{\mathcal{B}_H}[\log q_\phi(\Delta_t^H | \mathbf{s}_t, \tilde{\mathbf{a}}_t)]. \quad (6)$$

The proposal is kept as an explicit model input so that the Correction Model need not recover that high-dimensional action from the compact RL state. During data collection, small Gaussian exploration noise is added to $\mathbf{a}_\theta$ as in TD3 [37].

C. State-Dependent Primal-Dual Optimization

Introducing a nonnegative state-dependent multiplier $\lambda(\cdot) \geq 0$ for Eq. (4) yields the population Lagrangian

$$\begin{aligned} \mathcal{L}(\theta, \lambda) = \mathbb{E}_\mathcal{B} [& -Q_\psi(\mathbf{s}_t, \mathbf{a}_\theta) \\ & + \eta \|\mathbf{a}_\theta - \tilde{\mathbf{a}}_t\|_2^2 + \lambda(\mathbf{s}_t)(\rho_\phi - \varepsilon)]. \end{aligned} \quad (7)$$

with saddle problem $\max_{\lambda \geq 0} \min_{\theta} \mathcal{L}(\theta, \lambda)$. We approximate λ by $\lambda_\omega(\mathbf{s}_t)$ with a softplus output. For the actor step, $-\lambda_\omega \varepsilon$ is constant in θ and can be omitted. We further normalize each sampled state by $1 + \lambda_\omega(\mathbf{s}_t)$ [11], giving

$$\mathcal{L}_\pi(\theta) = \mathbb{E}_\mathcal{B} \left[\frac{-Q_\psi(\mathbf{s}_t, \mathbf{a}_\theta) + \eta \|\mathbf{a}_\theta - \tilde{\mathbf{a}}_t\|_2^2 + \lambda_\omega(\mathbf{s}_t) \rho_\phi}{1 + \lambda_\omega(\mathbf{s}_t)} \right]. \quad (8)$$

The normalization stabilizes large-multiplier updates.

Algorithm 1 BEE: Intervention-Adaptive Policy Optimization

Require: Frozen VLA π_{vla} ; constraint bound ε , anchor weight η , target rate τ

- 1: Initialize π_θ , q_ϕ , λ_ω , Q_ψ , and target critic $\tilde{\psi} \leftarrow \psi$; seed $\mathcal{B}_H$ with pre-collected corrections
 - 2: **for** each chunk boundary t **do**
 - 3: Obtain $(\mathbf{s}_t, \tilde{\mathbf{a}}_t)$ and compute $\mathbf{a}_\theta = \tilde{\mathbf{a}}_t + \Delta_\theta^\pi(\mathbf{s}_t, \tilde{\mathbf{a}}_t)$
 - 4: **if** human intervenes **then**
 - 5: Execute $\mathbf{a}_t^H$
 - 6: **else**
 - 7: Execute $\mathbf{a}_\theta$ with exploration noise
 - 8: **end if**
 - 9: Store the transition in $\mathcal{B}_R$ and, if corrected, store $(\mathbf{s}_t, \tilde{\mathbf{a}}_t, \mathbf{a}_t^H)$ in $\mathcal{B}_H$
 - 10: Update Q_ψ by Eq. (2); soft-update $\tilde{\psi} \leftarrow (1 - \tau)\tilde{\psi} + \tau\psi$
 - 11: Every M new corrections, update q_ϕ by NM steps on Eq. (6)
 - 12: Form $\hat{\mathbf{a}}_t^H$ and evaluate ρ_ϕ by Eq. (3)
 - 13: Update π_θ by Eq. (8) and λ_ω by Eq. (9)
 - 14: **end for**
-

The multiplier is trained by dual ascent, implemented as minimization of

$$\mathcal{L}_\lambda(\omega) = -\mathbb{E}_{\mathcal{B}}[\lambda_\omega(\mathbf{s}_t)(\rho_\phi - \varepsilon)]. \quad (9)$$

Human corrections therefore shape how the policy is constrained rather than serving as direct policy targets.

D. Training Procedure

The Correction Model is pre-trained on the corrections that seed $\mathcal{B}_H$ and then updated only after every M new intervention samples, for NM gradient steps at a fixed update-to-data ratio N . This conservative schedule reduces variance collapse on the small intervention buffer; full details appear in Section C. Algorithm 1 summarizes the complete online training loop.

V. EXPERIMENTS

A. Experimental Setup

a) Real-world tasks: We evaluate three real-world manipulation tasks, as demonstrated in Figure 2, each ending in a precision-critical contact phase in which small action errors determine success:

- **Phone charging** The left arm grasps and lifts the cable while the right arm approaches and grasps the charger, aligns it with the socket, and inserts it, which demands millimeter-level alignment under partial observability of the socket and the charger tip.
- **Snack hanging.** The robot approaches and grasps a deformable snack package, lifts it toward the rack, and hangs it by the hole in the package. The package deforms under the grasp, and the resulting pose shifts the target location across episodes.

- **Cloth aligning.** The robot grasps a piece of cloth, lifts it, moves it over the target frame, and aligns it onto that frame. Deformation during transport makes the final contact and release the decisive step.

Each task spans 30–60 s of execution, i.e., roughly 900–1800 control steps at 30 Hz.

b) Simulation task: We additionally evaluate **bowl placing** in LIBERO-PRO [38], an evaluation extension of the LIBERO benchmark [39], in which the arm grasps a bowl, moves it above the plate, and places it down over a comparable horizon. The benchmark configuration is detailed in Appendix A.

c) Implementation details: We fine-tune the open-source $\pi_{0.5}$ weights with behavior cloning on each task, then freeze the resulting VLA together with an RLT-style [10] latent-token encoder as the fixed VLA–RL interface. During online RL, only the lightweight Residual Policy, the critic, the Correction Model, and the state-dependent multiplier are trained, all off-policy at the chunk level under an action normalization shared across methods. The latter three are training-time components: at evaluation and deployment, interventions and exploration noise are switched off and the robot is driven by the Residual Policy on top of the frozen VLA interface, at no additional inference cost. All four tasks are long-horizon. For phone charging and cloth aligning, which span the longest horizons, online RL refines only their precision-critical phase, whereas the relatively shorter snack hanging and bowl placing are trained over the full task. The real-world robot is controlled at 30 Hz with a task-dependent action space: on phone charging, the per-timestep action is 16-dimensional (14 dual-arm joint dimensions, 7 per arm, plus 2 end-effector dimensions), and a chunk length of $K=10$. All tasks use a sparse task-completion reward, except cloth aligning with a vision-based dense reward. Further task, reward, and implementation details appear in Appendices A–C.

d) Baselines: We compare **BEE** against three baselines that operate in the same frozen-VLA setting. The Base Policy is the $\pi_{0.5}$ [5] VLA after supervised fine-tuning (SFT), evaluated before any online adaptation as a pre-adaptation reference. RLT [10] performs chunk-level online RL over the frozen VLA’s latent token, regularized toward the VLA proposal. DSRL [9] keeps the VLA fixed and steers it by searching its latent noise space. We additionally evaluate DAGger [23], SiLRI [11], and HIL-SERL [7] on a subset of our tasks and report them in Table IV of Appendix E.

e) Protocol and metrics: All methods start from the same fine-tuned VLA and are compared at a matched robot-data budget of around 20 pre-collected episodes of human corrections plus a fixed number of online episodes per task. RLT and **BEE**, the two methods that train with human intervention, are seeded with those 20 episodes and follow the same intervention protocol, whereas DSRL maintains no demonstration buffer and receives them as additional online interaction. The success rate is measured in three evaluation rounds of 20 trials per task and reported as the mean and standard deviation across rounds. Phone charging and cloth

TABLE I: **Comparison of BEE with the Base Policy, RLT, and DSRL.** Success rate (SR, %, $\uparrow$) and human intervention rate (Int., %, $\downarrow$) on the real-world tasks phone charging, snack hanging, and cloth aligning, and on the LIBERO-PRO simulation task bowl placing. The Base Policy and DSRL involve no online intervention, so no intervention rate is reported.

Method	Phone Charging		Snack Hanging		Cloth Aligning		Bowl Placing		Overall	
	SR $\uparrow$	Int. $\downarrow$	SR $\uparrow$	Int. $\downarrow$	SR $\uparrow$	Int. $\downarrow$	SR $\uparrow$	Int. $\downarrow$	SR $\uparrow$	Int. $\downarrow$
Base Policy	90.0 $\pm$ 5.0	—	0.0 $\pm$ 0.0	—	36.7 $\pm$ 7.6	—	28.3 $\pm$ 7.6	—	38.8	—
RLT	58.3 $\pm$ 2.9 [†]	22.5	21.7 $\pm$ 5.8	62.1	73.3 $\pm$ 7.6	85.3	76.7 $\pm$ 17.6	26.2	57.5	49.0
DSRL	93.3 $\pm$ 2.9	—	13.3 $\pm$ 2.9	—	31.7 $\pm$ 2.9	—	30.0 $\pm$ 5.0	—	42.1	—
BEE	100.0 $\pm$ 0.0	12.2	85.0 $\pm$ 0.0	17.0	90.0 $\pm$ 10.0	65.3	90.0 $\pm$ 13.2	27.4	91.2	30.5

[†]Reported at the matched online-data budget used for all methods. RLT requires roughly 3 $\times$ the online data to reach 96.7 $\pm$ 5.8.

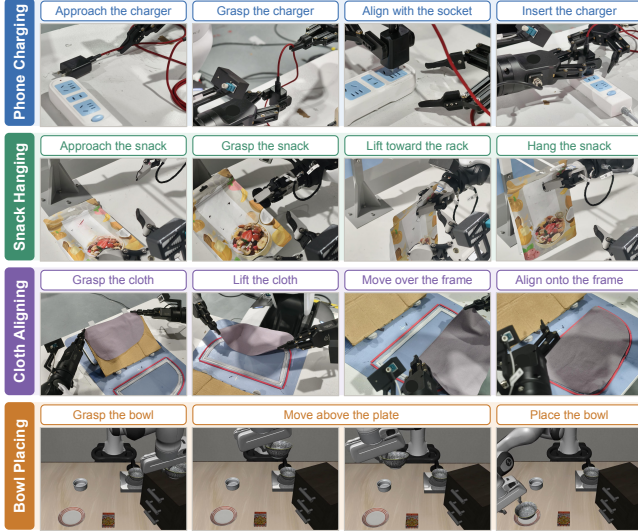


Fig. 2: **Evaluation Tasks.** Keyframes of the three real-world tasks (phone charging, snack hanging, cloth aligning) and of the LIBERO-PRO simulation task, bowl placing.

aligning report success only on their precision-critical phase, whereas snack hanging and bowl placing report full-task success. The human intervention rate is the proportion of control steps taken over by the human during online training. Full budget accounting and intervention details are provided in Appendix D.

B. Results and Analysis

a) *Overall comparison:* Table I compares **BEE** with the Base Policy, RLT, and DSRL across phone charging, snack hanging, cloth aligning, and bowl placing. **BEE** attains the highest success rate on every task and the lowest intervention rate on all three real-world tasks as well as overall. The gains hold across very different Base-Policy starting points. Where the Base Policy is already strong, **BEE** improves it further to a saturated success rate within the matched online-data budget, and where it starts near zero, **BEE** still reaches a comparably high success rate. The other methods improve far less under the same budget, in some cases even falling below the Base Policy, and RLT does so at a substantially higher intervention rate. Additional comparisons with DAgger, SiLRI,

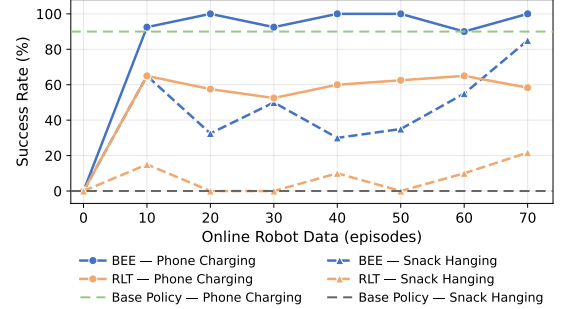


Fig. 3: **Sample Efficiency on Phone Charging and Snack Hanging.** Success rate against the number of online robot episodes, with color distinguishing **BEE** from RLT and line style distinguishing the two tasks; the horizontal dashed lines mark the Base Policy. The right end of the axis is the matched online-data budget at which Table I is reported. **BEE** stays above RLT at every budget on both tasks.

and HIL-SERL are reported in Table IV of Appendix E; none reaches competitive performance under the same budget.

b) *Sample efficiency:* Figure 3 tracks the success rate over online episodes on phone charging and snack hanging. On phone charging, where the Base Policy is already strong, **BEE** matches it within the first few episodes and then saturates. On snack hanging, where the Base Policy never succeeds, **BEE** first explores away from it, which temporarily lowers the success rate before it converges to a high one well inside the same budget. RLT improves early on phone charging but then plateaus below the Base Policy, and on snack hanging it stays close to the Base Policy throughout. The two methods also differ in how the robot behaves while learning. Because **BEE** anchors every update to the VLA proposal and tightens the correction constraint only along the dimensions humans correct consistently, its motion stays smooth and human-like from the first episodes onward, in training as in evaluation, whereas RLT visibly jitters early in training before settling. Smoother early behavior in turn calls for fewer human corrections, and **BEE** indeed attains the lowest intervention rate on all three real-world tasks, as reported in Table I. Taken together, these differences reflect how efficiently each method turns online robot data and human corrections into usable behavior.

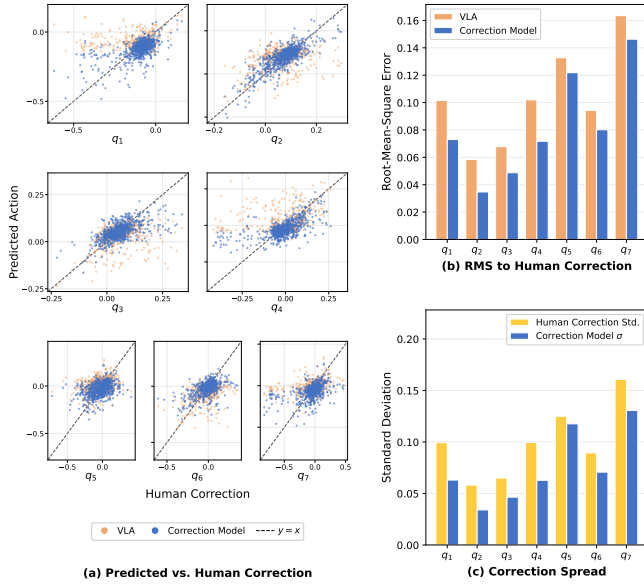


Fig. 4: Correction Model Calibration on Phone Charging. evaluated on human-intervention demonstrations over the seven joint dimensions q_1 – q_7 of the intervened arm. **(a)** The VLA proposal $\hat{\mathbf{a}}_t$ and the corrected action $\hat{\mathbf{a}}_t^H$ produced by the Correction Model, plotted against the human correction. Points closer to the $y=x$ line match it better. **(b)** Per-dimension root-mean-square error to the human correction. The corrected action reduces this error below the VLA proposal in every dimension. **(c)** Predicted per-dimension standard deviation of the Correction Model and empirical standard deviation of human corrections. The prediction follows the same per-dimension pattern as the empirical spread.

c) Per-dimension reliability of human corrections:

Dimension-wise weighting assumes that human corrections are reliable on some action dimensions but noisy on others. We test this on held-out human-intervention demonstrations, restricted to the seven joint dimensions of the intervened arm (q_1 – q_7), because the other arm and the end-effector receive essentially no human correction. As Figure 4c shows, the empirical spread of the corrections indeed varies by about $3\times$ across dimensions, from the most consistently edited (q_2) to the most variable (q_7). The corrections are nonetheless predictable from the state and the VLA proposal. Recentered by the Correction Model, the corrected action lies closer to the $y=x$ line than the VLA proposal and reduces the per-dimension error to the human correction in every dimension, as Figure 4a,b illustrates. Its predicted uncertainty σ follows the same per-dimension pattern as the empirical spread, which is what the constraint requires, since only the ratios of the predicted variances set the per-dimension weighting.

d) Dimension-wise constraint allocation: Figure 5 then shows how the learned uncertainty reshapes policy optimization, comparing the dimension-wise Mahalanobis constraint used by **BEE** with the state-wise constraint, an otherwise-identical variant of comparable overall tightness. The state-

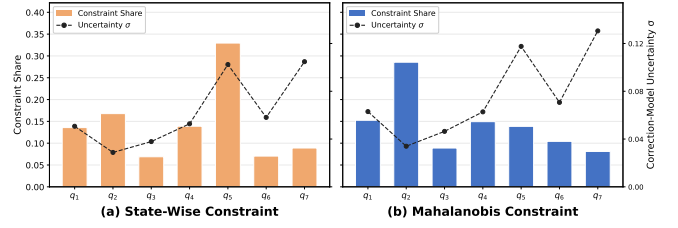


Fig. 5: Dimension-wise Constraint Allocation on Phone Charging. Bars show each dimension’s constraint share (its normalized contribution to ρ_ϕ), and the overlaid curve is the predicted uncertainty σ of the Correction Model. **(a)** The state-wise form weights all dimensions equally, so a high-uncertainty dimension such as q_5 claims the largest share. **(b)** The Mahalanobis form concentrates the share on low-uncertainty, consistently corrected dimensions (e.g., q_2) and downweights high-uncertainty ones (e.g., q_5).

wise form applies a single per-state constraint bound and measures deviation by unweighted squared error, so each dimension’s share follows the raw magnitude of $(\mathbf{a}_\theta - \hat{\mathbf{a}}_t^H)^2$: the largest share by far goes to q_5 , one of the least consistently corrected dimensions. Normalizing by the predicted variances redistributes this budget: under the Mahalanobis form the largest share moves to q_2 , the most consistently corrected dimension, the share of q_5 drops by more than half, and the most variable dimension q_7 receives the smallest share of all. This is exactly the design intent: along q_2 the correction constraint dominates and pulls the Residual Policy tightly toward the corrected action, whereas along the variable dimensions it relaxes and value maximization together with the VLA anchor loss drive the update. The state-level complement of this dimension-wise mechanism, the training dynamics of the state-dependent multiplier $\lambda_\omega(\mathbf{s}_t)$, is shown in Figure 7 of Appendix F.

C. Ablations

Figure 6 reports ablations of **BEE** on phone charging and snack hanging. The two variants that touch the Correction Model are the weakest: they hold the two lowest success rates on snack hanging and the two highest intervention rates on phone charging, where the saturated task leaves little headroom in success rate. *w/o Correction Model* supervises the actor with raw corrections, which are too inconsistent to serve as policy targets. The gap is largest on snack hanging, where the Base Policy starts from a near-zero success rate. *w/o VLA Proposal in Correction Model* is weaker still on that task: because the correction residual is defined relative to the proposal it overrides, conditioning on the state alone leaves the constraint center carrying the sampling deviation of the proposal instead of cancelling it, and lets the predicted variances rank dimensions by how variable the VLA is rather than by how consistently humans correct them. On snack hanging, modeling corrections without the proposal is thus worse than not modeling them at all. The constraint geometry matters as much: *w/o Mahalanobis Constraint* matches **BEE** on the saturated phone charging task but

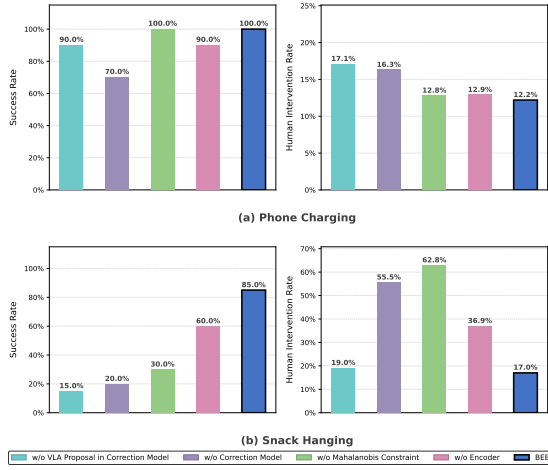


Fig. 6: **Ablation of Human Correction Modeling** on (a) phone charging and (b) snack hanging, each reporting success rate (left) and human intervention rate (right) for *w/o VLA Proposal in Correction Model* (the Correction Model conditioned on s_t alone), *w/o Correction Model* (direct actor supervision from raw human corrections), *w/o Mahalanobis Constraint* (a single state-wise constraint in place of the dimension-wise one), *w/o Encoder* (mean-pooled frozen VLA features in place of the latent-token encoder), and **BEE**.

falls far short on snack hanging, where the dimension-wise constraint sharply raises success and cuts the intervention rate—the per-dimension geometry is decisive exactly where a single scalar is not. Finally, *w/o Encoder* is the strongest of the ablated variants on snack hanging and stays close to **BEE** on phone charging, so the gains stem from the proposal-conditioned Correction Model and the dimension-wise constraint rather than from the token interface, and **BEE** remains strong even without training a dedicated encoder.

VI. CONCLUSIONS

We presented **BEE**, an intervention-adaptive framework for VLA-anchored real-world RL that converts human corrections of VLA proposals into an uncertainty-aware constraint on policy optimization, allowing the policy to move beyond expert imitation. At a matched online-data budget on three real-world manipulation tasks and a LIBERO-Pro simulation task, **BEE** attains the highest success rate on every task, 91.2% on average against 57.5% for RLT and 42.1% for DSRL, and the lowest human intervention rate on all real-world tasks. Our ablations show that both the *Correction Model* and the dimension-wise geometry of the constraint are necessary, and our analysis confirms that the learned uncertainty shifts the constraint onto the dimensions humans correct consistently. However, the predicted uncertainty is less reliable where human corrections are sparse, and extending to multi-phase tasks, multiple operators, and broader embodiments is a natural next step. We see modeling the structure of human corrections, rather than imitating them, as a promising direction for turning scarce real-world supervision into a reusable prior for VLA-anchored RL.

REFERENCES

- [1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.15818>
- [2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, *et al.*, “OpenVLA: An open-source vision-language-action model,” 2024.
- [3] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” in *Robotics: Science and Systems XXI*, 2025.
- [4] NVIDIA, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. J. Fan, Y. Fang, D. Fox, F. Hu, *et al.*, “GR00T N1: An open foundation model for generalist humanoid robots,” 2025.
- [5] Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.16054>
- [6] Physical Intelligence, “ $\pi_{0.6}^*$: A VLA that learns from experience,” 2025.
- [7] J. Luo, C. Xu, J. Wu, and S. Levine, “Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning,” *Science Robotics*, vol. 10, no. 105, p. eads5033, 2025.
- [8] K. Lei, H. Li, D. Yu, Z. Wei, L. Guo, Z. Jiang, Z. Wang, S. Liang, and H. Xu, “RL-100: Performant robotic manipulation with real-world reinforcement learning,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.14830>
- [9] A. Wagenmaker, M. Nakamoto, Y. Zhang, S. Park, W. Yagoub, A. Nagabandi, A. Gupta, and S. Levine, “Steering your diffusion policy with latent space reinforcement learning,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.15799>
- [10] C. Xu, J. T. Springenberg, M. Equi, A. Amin, A. Esmail, S. Levine, and L. Ke, “RL Token: Bootstrapping online RL with vision-language-action models,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.23073>
- [11] Y. Zhao, H. Jin, L. Jiang, X. Zhang, K. Wu, P. Ren, Z. Xu, Z. Che, L. Sun, D. Wu, C. H. Liu, and J. Tang, “Real-world reinforcement learning from suboptimal interventions,” 2025. [Online]. Available: <https://arxiv.org/abs/2512.24288>
- [12] G. Lu, W. Guo, C. Zhang, Y. Zhou, H. Jiang, Z. Gao, Y. Tang, and Z. Wang, “VLA-RL: Towards masterful and general robotic manipulation with scalable reinforcement learning,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.18719>
- [13] S. Tan, K. Dou, Y. Zhao, and P. Krähennühl, “Interactive post-training for vision-language-action models,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.17016>
- [14] J. Hu, R. Hendrix, A. Farhadi, A. Kembhavi, R. Martín-Martín, P. Stone, K.-H. Zeng, and K. Ehsani, “FLaRe: Achieving masterful and adaptive robot policies with large-scale reinforcement learning fine-tuning,” in *IEEE International Conference on Robotics and Automation*, 2025, pp. 3617–3624.
- [15] Z. Zhang, K. Zheng, Z. Chen, J. Jang, Y. Li, S. Han, C. Wang, M. Ding, D. Fox, and H. Yao, “GRAPE: Generalizing robot policy via preference alignment,” 2024. [Online]. Available: <https://arxiv.org/abs/2411.19309>
- [16] C. Xu, Q. Li, J. Luo, and S. Levine, “RLDG: Robotic generalist policy distillation via reinforcement learning,” 2024. [Online]. Available: <https://arxiv.org/abs/2412.09858>
- [17] Y. Chen, S. Tian, S. Liu, Y. Zhou, H. Li, and D. Zhao, “ConRFT: A reinforced fine-tuning method for VLA models via consistency policy,” in *Robotics: Science and Systems XXI*, 2025.
- [18] Y. Guo, J. Zhang, X. Chen, X. Ji, Y.-J. Wang, Y. Hu, and J. Chen, “Improving vision-language-action model with online reinforcement learning,” in *IEEE International Conference on Robotics and Automation*, 2025, pp. 15 665–15 672.
- [19] A. Nair, A. Gupta, M. Dalal, and S. Levine, “AWAC: Accelerating online reinforcement learning with offline datasets,” 2020. [Online]. Available: <https://arxiv.org/abs/2006.09359>
- [20] S. Fujimoto and S. S. Gu, “A minimalist approach to offline reinforcement learning,” in *Advances in Neural Information Processing Systems*, 2021, pp. 20 132–20 145.

- [21] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling, “Residual policy learning,” 2018. [Online]. Available: <https://arxiv.org/abs/1812.06298> 2
- [22] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, “Residual reinforcement learning for robot control,” in *IEEE International Conference on Robotics and Automation*, 2019, pp. 6023–6029. 2
- [23] S. Ross, G. J. Gordon, and J. A. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 627–635. [Online]. Available: <https://proceedings.mlr.press/v15/ross11a.html> 2, 4, 10
- [24] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, “HG-Dagger: Interactive imitation learning with human experts,” in *IEEE International Conference on Robotics and Automation*, 2019, pp. 8077–8083. 2
- [25] R. Hoque, A. Balakrishna, E. Novoseller, A. Wilcox, D. S. Brown, and K. Goldberg, “ThriftyDagger: Budget-aware novelty and risk gating for interactive imitation learning,” in *Conference on Robot Learning (CoRL)*, 2021, pp. 598–608. 2
- [26] A. Mandlekar, D. Xu, R. Martín-Martín, Y. Zhu, L. Fei-Fei, and S. Savarese, “Human-in-the-loop imitation learning using remote teleoperation,” 2020. [Online]. Available: <https://arxiv.org/abs/2012.06733> 2
- [27] H. Liu, S. Nasiriany, L. Zhang, Z. Bao, and Y. Zhu, “Robot learning on the job: Human-in-the-loop autonomy and learning during deployment,” in *Robotics: Science and Systems XIX*, 2023. 2
- [28] J. Luo, P. Dong, Y. Zhai, Y. Ma, and S. Levine, “RLIF: Interactive imitation learning as reinforcement learning,” in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: <https://openreview.net/forum?id=oLLZhBBSOU> 2
- [29] Z. Peng, W. Mo, C. Duan, Q. Li, and B. Zhou, “Learning from active human involvement through proxy value propagation,” in *Advances in Neural Information Processing Systems*, 2023, pp. 77969–77992. 2
- [30] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, “Efficient online reinforcement learning with offline data,” in *International Conference on Machine Learning (ICML)*, 2023, pp. 1577–1594. 2
- [31] J. Luo, Z. Hu, C. Xu, Y. L. Tan, J. Berg, A. Sharma, S. Schaal, C. Finn, A. Gupta, and S. Levine, “SERL: A software suite for sample-efficient robotic reinforcement learning,” in *IEEE International Conference on Robotics and Automation*, 2024, pp. 16961–16969. 2
- [32] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Robotics: Science and Systems XIX*, 2023. 2
- [33] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Robotics: Science and Systems XIX*, 2023. 2
- [34] D. A. Nix and A. S. Weigend, “Estimating the mean and variance of the target probability distribution,” in *IEEE International Conference on Neural Networks (ICNN)*, vol. 1, 1994, pp. 55–60. 2
- [35] A. Kendall and Y. Gal, “What uncertainties do we need in bayesian deep learning for computer vision?” in *Advances in Neural Information Processing Systems*, 2017, pp. 5574–5584. 2
- [36] C. Tessler, D. J. Mankowitz, and S. Mannor, “Reward constrained policy optimization,” in *International Conference on Learning Representations (ICLR)*, 2019. 3
- [37] S. Fujimoto, H. van Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *International Conference on Machine Learning (ICML)*, 2018, pp. 1587–1596. 3
- [38] X. Zhou, Y. Xu, G. Tie, Y. Chen, G. Zhang, D. Chu, P. Zhou, and L. Sun, “LIBERO-PRO: Towards robust and fair evaluation of vision-language-action models beyond memorization,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.03827> 4, 8
- [39] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2023. [Online]. Available: <https://arxiv.org/abs/2306.03310> 4, 8

A. Task and Environment Details

The main paper introduces the three real-world manipulation tasks, phone charging, snack hanging, and cloth aligning, and the simulated bowl placing task in LIBERO-Pro [38]. Table II summarizes how the four differ, and the subsections below give the platform, the action spaces, and the details of the simulated task.

1) Robot Platform, Observations, and Actions:

a) *Platform and observations:* The real-world tasks run on a dual-arm robot. Three 480×640 RGB cameras observe the workspace, one on the head and one on each wrist. The frozen VLA consumes these views as $\mathbf{o}_t$, the language instruction g , and the proprioceptive state $\mathbf{p}_t$, which holds the arm joint states and the effector states, whereas the online learner observes only the latent token $\mathbf{z}_t$ produced by the frozen extractor together with $\mathbf{p}_t$. The dense reward for cloth aligning is computed from the head-camera view.

b) *Control frequency and action space:* Table II gives the per-task values, which differ in both the action representation and the learner chunk length. Phone charging and cloth aligning are the two dual-arm tasks and are controlled in joint space, as fourteen dual-arm joint dimensions together with two further end-effector dimensions, and online RL is allowed to edit only the joint dimensions of the right arm, which performs the precision-critical step and is also the arm the operator intervenes on. Snack hanging and bowl placing are instead controlled by end-effector commands, and online RL edits the pose dimensions while the gripper is held at the VLA proposal.

2) *Simulation Task: Bowl Placing:* Bowl placing is task 9 of the `libero_spatial` suite of LIBERO-Pro [38], an evaluation extension of the LIBERO benchmark [39].

B. Reward Functions

Phone charging, snack hanging, and bowl placing use a sparse task-completion reward, and cloth aligning uses a vision-based dense reward.

a) *Sparse task-completion reward:* For phone charging, snack hanging, and bowl placing, the per-step reward is $r_t = 0$ throughout the episode and $r_t = 1$ only when the task is judged complete, after which the episode terminates.

b) *Vision-based dense reward for cloth aligning:* We derive the reward from vision, as the increment of an alignment potential between the cloth and the target region. The same perception module also defines the success criterion for this task, since a trial counts as a success when the intersection over union of the cloth mask and the target-region mask exceeds 0.85 at some point in the episode.

C. Implementation Details

Table III gives the hyperparameters of the phone-charging configuration, the run analyzed throughout the main paper; the per-task control rates, action dimensions, chunk lengths, and online budgets are listed in Table II. The paragraphs below situate those values against the method of the main paper.

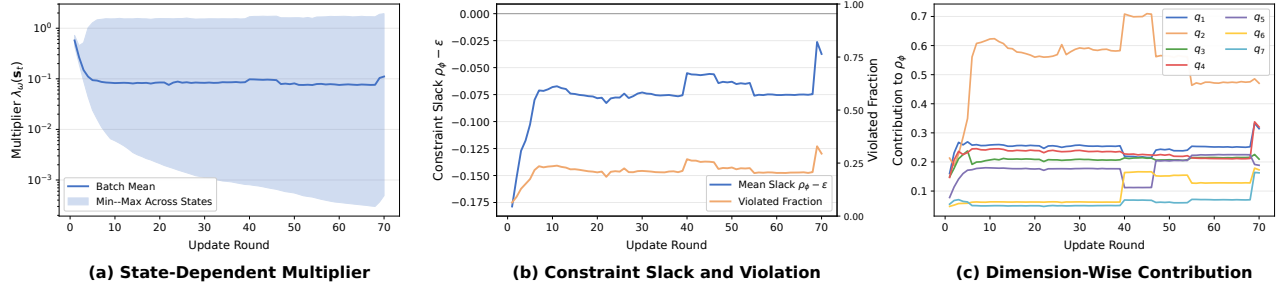


Fig. 7: **Training Dynamics of the State-Dependent Multiplier** on phone charging. (a) State-wise multiplier statistics. (b) Constraint slack and violation rate. (c) Per-dimension contributions to ρ_ϕ .

	Phone charging	Snack hanging	Cloth aligning	Bowl placing
Environment	real	real	real	LIBERO-Pro
Control rate	30 Hz	30 Hz	30 Hz	20 Hz
Action dims d / D	16 / 160	8 / 160	16 / 160	7 / 35
VLA chunk length (approach)	50	50	50	10
Learner chunk length K	10	20	10	5
Trained by online RL	critical phase	full task	critical phase	full task
Success reported on	critical phase	full task	critical phase	full task
Reward	sparse completion	sparse completion	dense, SAM 3	sparse completion
Completion signal	operator button	operator button	mask IoU	operator button
Success criterion	charger inserted	hung on rack	mask IoU > 0.85	bowl on plate
Online episode budget	70	70	70	25

TABLE II: **Per-Task Specification.** The action space lists the per-timestep dimension d and the chunked dimension $D = Kd$ seen by the actor; the subset of dimensions online RL is allowed to edit, with the remaining dimensions held at the VLA proposal, is described in the text. On phone charging and cloth aligning online RL refines the precision-critical phase only, so an online episode covers that phase and success is reported on it. The online episode budget is the one given to the methods that are seeded with pre-collected human corrections. Section D gives the budget of the methods that are not.

a) Backbone and frozen interface: All experiments start from the open-source $\pi_{0.5}$ weights [5], fine-tuned with behavior cloning on each task separately and then frozen, so the frozen VLA is task-specific. The extractor that maps its internal features to the latent token $\mathbf{z}_t$ is an RLT-style latent-token encoder [10], a four-layer encoder and four-layer decoder of width 2048 with 16 attention heads, so $\mathbf{z}_t$ has dimension 2048. It is frozen as well, so the interface pair $(\mathbf{s}_t, \tilde{\mathbf{a}}_t)$ stays fixed throughout online training.

b) Hardware and compute: Evaluation overlaps inference with execution by real-time chunking, on the platform described in Section A. Over its episode budget, one online run of **BEE** performs between roughly 20,000 and 90,000 gradient updates depending on the task, since the update-to-data ratio is fixed while the episode length and the online budget are not. Training length is measured in online episodes throughout rather than in wall-clock time, so a run ends when its episode budget is exhausted. The learner and VLA inference run on a single RTX 4090D for every method except DAGger, which was trained on an A100.

D. Intervention Protocol and Evaluation Details

a) Matched robot-data budget: Every method is given the same total amount of interaction with the robot, which is 90 episodes on each of the three real-world tasks and 45 on bowl placing. What differs is how that total is

split between episodes collected before online training and episodes collected during it. **BEE**, **RLT**, **SiLRI**, and **HIL-SERL** maintain a demonstration buffer, so around 20 of their episodes are collected in advance as human corrections and seed $\mathcal{B}_H$, leaving the online budget of 70 episodes on the real-world tasks and 25 on bowl placing that Table II lists. **DSRL** and **DAGger** maintain no demonstration buffer, and seeding them with one would depart from their original formulations, since both already start from the same fine-tuned VLA that was trained on task demonstrations. They therefore spend those 20 episodes online instead, for 90 online episodes on the real-world tasks and 45 on bowl placing. Reported success rates are read at the end of this budget for every method.

b) Intervention protocol: During online training a human operator monitors execution and takes over whenever the robot is about to fail in a way it cannot recover from on its own, which in practice concentrates the corrections on the precision-critical contact phase of each task. In simulation the operator instead uses a 6-DoF SpaceMouse, with two buttons for the gripper.

c) Phase switching: For the two-phase tasks, phone charging and cloth aligning, **RLT** and **BEE** switch from the frozen VLA approach to online RL using a progress head trained during SFT, entering the precision-critical phase once the predicted progress crosses a threshold.

Hyperparameter	Value	Hyperparameter	Value
<i>Interface and action space</i>		<i>Correction Model</i>	
Control frequency	30 Hz	Correction Model batch size	256
VLA chunk length, approach / online	50 / 10	Update interval M (samples)	100
Learner chunk length K	10	Update-to-data ratio N	0.12
Per-timestep action dim. d	16	Constraint bound ε	0.3
Chunked action dim. $D = Kd$	160	Per-dimension variance floor	0.02
RL-editable dimensions	7 arm joints	VLA anchor weight η	1.0
Action normalization	$\pi_{0.5}$ z-score	<i>Data and exploration</i>	
<i>Networks and optimization</i>		Seed episodes in $\mathcal{B}_H$	20
MLP depth / width (all modules)	3 / 512	$\mathcal{B}_H$ -to- $\mathcal{B}_R$ sampling ratio	1:1
Critic heads	2, min over heads	Reference-action dropout	0.5
Discount γ	0.99	VLA warmup before handover	5 weight syncs
Target update rate τ	0.005		
Update-to-data ratio (critic, policy)	5		
Critic updates per policy update	2		
Batch size	256		
Optimizer	Adam(0.9, 0.95, $\varepsilon=10^{-8}$)		
Learning rate (θ, ψ, ϕ)	3×10^{-4}		
Multiplier learning rate (ω)	3×10^{-6}		
Gradient-norm clip	1.0		
Exploration noise σ_{expl}	0.01		

TABLE III: **Hyperparameters of BEE on Phone Charging.** Values are those of the run analyzed in the main paper; the quantities that vary by task are listed per task in Table II. All distances, thresholds, and noise scales are expressed in normalized action units.

E. Baseline Implementations

Because the comparisons are conducted on a real robot, all baselines are implemented under a common setup. Every method is trained and evaluated on the same tasks with the same reward source, the same VR interface for human interventions and completion signals, the same action normalization, and the same matched robot-data budget and evaluation protocol described above. The methods that build on a frozen VLA start from the identical fine-tuned $\pi_{0.5}$ checkpoint and, where they use a latent-token interface, from the identical frozen extractor. SiLRI and HIL-SERL are the exception, since their original formulations do not assume a pretrained VLA, and we retain that setting rather than reworking them into ours. Following their published implementations, both encode the camera views with a ResNet-10 initialized from the publicly released pretrained weights and train the policy and the critic on top of it, so what is trained from scratch in these two baselines is the policy rather than the visual representation. The paragraphs below give the per-method details.

a) Base Policy: The number of demonstrations used for supervised fine-tuning differs by task and ranges from 200 to 900 episodes, so the online budget every method is then given is a small fraction of the data the shared starting point was trained on. The Base Policy consumes none of that budget and is also the initialization of the frozen-VLA methods.

b) RLT: RLT [10] has no public implementation and is reimplemented on the shared chunk-level actor-critic backbone, so it uses the same frozen VLA interface and the same action envelope as **BEE** and the comparison isolates how the two methods use human corrections. Following the original RLT rollout procedure, when the human intervenes,

Method	SR (%) $\uparrow$	Int. (%) $\downarrow$
<i>Phone charging</i>		
Base Policy	90.0 $\pm$ 5.0	–
DAGger	63.3 $\pm$ 5.8	7.8
BEE	100.0 $\pm$ 0.0	12.2
<i>Bowl placing</i>		
SiLRI	0.0	90.0+
HIL-SERL	0.0	90.0+
BEE	90.0 $\pm$ 13.2	27.4
<i>Snack hanging</i>		
SiLRI	0.0	90.0+
HIL-SERL	0.0	90.0+
BEE	85.0 $\pm$ 0.0	17.0

TABLE IV: **Comparison with DAGger, SiLRI, and HIL-SERL** under the matched robot-data budget. Each method is evaluated on the task where the comparison is informative. Intervention rates follow the accounting in Section D and are not directly comparable across methods.

the human correction replaces the VLA proposal stored for that transition.

c) DSRL: DSRL [9] acts on the same frozen fine-tuned VLA and is run in the chunk-10 configuration of the original formulation: noise is sampled for the first 10 steps of a chunk, the last noise vector is repeated to fill the 50-step VLA chunk, and the first ten actions of the result are executed.

d) DAGger: DAGger [23] is applicable on top of the same fine-tuned VLA, and we evaluate it on phone charging, where its interactive-expert assumption applies directly. It lands below both **BEE** and the Base Policy it starts from, even though it receives 20 additional online episodes in place of the seed corrections, so aggregating and imitating the operator’s corrections degrades the very policy it was meant to improve.

e) *SiLRI and HIL-SERL*: Neither is competent enough at this budget to be left running, so the operator holds control for almost the entire episode throughout, and neither run makes progress without that continuous intervention.

f) *Scope of the comparison*: Table IV compares **BEE** with DAGger, SiLRI, and HIL-SERL under the same intervention protocol and robot-data budget as in the main paper. Each is evaluated on the task where the comparison is informative. SiLRI and HIL-SERL train task-specific visuomotor policies from scratch, and all of our tasks span roughly a minute of continuous control, so a budget of tens of episodes is far below what a from-scratch policy needs on any of them.

F. Training Dynamics of the State-Dependent Multiplier

The main paper analyzes how the dimension-wise constraint allocates its budget across action dimensions. Figure 7 complements that snapshot with the state-level behavior of the constraint during training, and three readings matter. The multiplier does not converge to one value: it stays near zero on states where the policy already agrees with the corrected action and large on the subset of states where the constraint is active, so a single global multiplier would have to average over two regimes that the state-dependent parameterization keeps apart. The violation rate settles well above zero rather than vanishing, which is what an interactive setting should produce, since newly collected corrections keep introducing states where the policy deviates from the corrected action and the multiplier re-tightens locally on them. And the per-dimension decomposition of ρ_ϕ reproduces the allocation the main paper reads off a single snapshot, so that mechanism persists throughout online training rather than holding at one point in it.